1. Warm-Up Review

```
#What do these lines print?
print("5" + "3") #str + str => concatenation 53
print(5 + 3) #int + int = int... 8
```

```
53
8
```

What Did You Learn last week?

Try to answer the following questions. You can write your answers in comments or discuss them with a partner.

1. What does the `input()` function do?

2. What is a variable? Why do we use it?

3. What happens if you do this?

```python age = input("Enter your age: ") print(age + 5)

```
age = input("Enter your age: ") #str
print(age + 5)
```

```
Enter your age: 15
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-919057698.py in <cell line: 0>()
      1 age = input("Enter your age: ") #str
----> 2 print(age + 5)

TypeError: can only concatenate str (not "int") to str
```

```
age = "17"
int(age)
```

```
17
```

```
# how do we fix this?
age = int(input("Enter your age: "))
type(age)
```

```
Enter your age: 15
```
```

```
int

age = int(input("Enter your age: "))

Enter your age: 56

'''
1. what are the different data types #int, float, bool, str, list,
None
2. what is the data type for each of the following:
    a. 7.6 #float
    b. True #bool
    c. 50 #int
    d. [2, 4, 6] #list
    e. 'Hello' #str
'''

-10/3

-3.3333333333333335

-10 // 3 #-3 or -4

-4

10 //3

3

'''
1. What are the names of the following operations:
    a. / #division
    b. * #multiplicaiton
    c. ** #exponentation
    d. // #integer division
    e. % #remainder
2. What is the output for the following:
    a. 20 / 3 = 6.67
    b. 20 // 3 = 6
    c. 20 % 3 = 2
    d. 20 ** 3 = 8000
    e. 3 - 4 * 5 % 3 = 3 - 20 % 3 = 3 - 2 = 1

3 - 20 % 3 = 3 - 2 = 1

    f. 40 / 5 + 30 % 8 = 8 + 30 % 8 = 8 + 6 = 14
'''

'''
1. which of the following is equivalent to x = x + 3
    a. x =+ 3
    b. x += x + 3
```

```
    c. x += 3
    d. x =+ x + 3
2. what is the short form of the following:
    a. x = x * 3   # x *= 3
    b. x = x / 3 # x /= 3
    c. x = x ** 3  # x **=3

'''

a = 3
b = 4
a = b # a = 4
a
```

```
4
```

```
a = 3
b = 4
a == b #comparing.. checking for equality
```

```
False
```

```
# =, a = b
# ==, a == b

'''
1. what's the output for:
    a. True or False  # True
    b. False or False # False
    c. True and False #False
    d. False and False #false
    e. True and True #True
    f. (3 == 5) or (4 == 4)  # False or True = True  ... False and
True = False
2. what's concatenation # string addition
3. what's the result for:
    a. "Welcome" + "to" + "Durame"  #WelcometoDurame
    b. "Welcome" + " " + "to" + "Durame" #Welcome toDurame
    c. "Welcome" + " " + "to" + " " + "Durame"  #Welcome to Durame
    d. "welcome" + 1 # TypeError: str + int: not allowed
'''

"Welcome" + "to" + "Durame"
```

```
{"type":"string"}
```

```
"Welcome" + " " + "to" + "Durame"
```

```
{"type":"string"}
```

Fix the Bug

```python

# Can you fix this code?

# This code asks for someone's age and tries to do math.

# But it will break! Fix it so it works correctly.

age = input("Enter your age: ") print("In 10 years, you will be " + age + 10)

```python
#write your fixed code here
age = int(input("How old are you? "))
print("IN 10 year, you will be " + str(age + 10))

# Your Turn: Use input() and math

# Ask the user for the number of apples they have
numberOfApples = int(input("How many apples you have? "))
# Ask how many more they will buy
more = int(input("how many more you wil buy? "))
# Calculate and print the total number of apples
print(numberOfApples + more)
# Write your code below:
```

2. Why Use Variables?

You already learned how to store values in variables. Now let’s go one step further.

We use variables to:

- Save user input
- Reuse values later
- Perform calculations
- Make programs more readable and useful

```

Real-World Example:

```python money = 50 lunch = 15 left = money - lunch print("Money left after lunch:", left)

```
# Area of circle with radius of 45.69603570572070m
Area = 3.14159265359 * 45.69603570572070**2
Area
```

```
6560.04657679078
```

```
pi = 3.14159265359
radius = 45.69603570572070
Area = pi*radius**2
Area
```

```
6560.04657679078
```

```
# Let's use variables to do math with money

# Ask how much money the user has
# Write your code here:
money = float(input("how much money you have? "))

# Ask how much lunch costs
# Write your code here:
lunch = float(input("how much is the lunch? "))

# Subtract lunch from money
# Write your code here:
left = money - lunch

# Print how much is left
# Write your code here:
print("you are left with ", left)

# Your Turn: Trip Budget

# Ask the user how much money they have
# Ask how much they spend on transport
# Ask how much they spend on snacks
# Calculate how much is left
# Print a report

# Write your code below:
```
```

3. Math Practice with Multiple Inputs

Practice: Use Input and Math to Solve Real Problems

Let's practice writing programs that:

- Ask for more than one input
- Use math (`+`, `-`, `*`, `/`) to calculate things
- Use variables to store and organize the answers

`Exercise 1`

```
# Calculate Total Cost

# Ask how much one sandwich costs
# sandwich_price = ...

# Ask how many sandwiches the user wants
# quantity = ...

# Multiply to get total
# total = ...

# Print the result
# print("Total cost:", total)
```

`Exercise 2`

```
# Calculate Average

# Ask for score on Test 1
# score1 = ...

# Ask for score on Test 2
# score2 = ...

# Ask for score on Test 3
# score3 = ...

# Calculate average
# average = ...

# Print the result
# print("Your average score is:", average)
```

4. More on Lists

Lists: A Box of Many Things

A **list** is a collection of items stored in a single variable.

Example: ```python fruits = ["apple", "banana", "cherry"]

`lists can contain any type of variable and also as many variable as you wish`

```
l = [2, 'a', True] # what is the data type of 2, 'a', and True
l

[2, 'a', True]
```

`we can also add a list into a list`

```
l = [ 2, 3, 4, [5, 6, 10], 2]
l

[2, 3, 4, [5, 6, 10], 2]

mylist = ["hello", False, None, 2.3572048, [34, 89, "pi"], 90]
mylist

['hello', False, None, 2.3572048, [34, 89, 'pi'], 90]
```

Adding Items into List

we use `.append()` function to add element at the end of the list

```
mylist = [1, 2, 3]
# so we want to add an element 4 in our list mylist
mylist.append(4)  # this is going to add the element 4 at the end of the list
mylist

[1, 2, 3, 4]

mylist.append("Error") #what's gonna be in the list now? [1, 2, 3, 4, "Error"]

mylist

[1, 2, 3, 4, 'Error']
```

we use `.extend()` to add items of another list to other list

```
mylist = [1, 2, 3]
list2 = [4, 6, 7]

mylist + list2
```

```
[1, 2, 3, 4, 6, 7]
```

```
mylist = [1, 2, 3]
list2 = [4, 6, 7]
#if we want to add elements of list2 to mylist, to make mylist = [1,
2, 3, 4, 6,7]
#let's use extend
mylist.extend(list2)

mylist
```

```
[1, 2, 3, 4, 6, 7]
```

```
mylist = [1, 2, 3]
list2 = [4, 6, 7]
#what will happen if we just use append?
mylist.append(list2)
mylist
```

```
[1, 2, 3, [4, 6, 7]]
```

```
#be carefull in using append() for adding list into list
mylist = [1, 2, 3]
list2 = [4, 6, 7]
mylist.append(list2)
print(mylist) #as you can see, the list is entirelly added to the
list, not individual elements
```

```
[1, 2, 3, [4, 6, 7]]
```

```
kebron = [2, 90]
kebron.extend([39, 100, 4])
kebron
```

```
[2, 90, 39, 100, 4]
```

```
kebron = [2, 90]
kebron.append([39, 100, 4]) #[2, 90, [39, 100, 4]]
kebron
```

```
[2, 90, [39, 100, 4]]
```

we use `.insert()` to add an element at specific index not at the end

```
# index is location of an element in a list
# .insert(location/index, value)
```

```
mylist = [1, 2, 3] #1 => index 0, 2 => index 1, 3 => index 2
mylist.insert(1, 5) #here we are saying, add number 5 at index 1
print(mylist) #mylist will look like [1, 5, 2, 3] ,because index 1
becomes 5
```

```
[1, 5, 2, 3]
```

```
mylist = [1, 2, 3] #1 => index 0, 2 => index 1, 3 => index 2
mylist.insert(2, 5) #here we are saying, add number 5 at index 2
print(mylist) #mylist will look like [1, 2, 5, 3] ,because index 2
becomes 5
```

```
[1, 2, 5, 3]
```

`Exercise`

```
#given the following
numbers = []
strings = []
#add the numbesr 1, 2 and 3 into the numbers list using append
function
numbers.append(1)
numbers.append(2)
numbers.append(3)
#add the words "welcome", "to", "Ethiopia" into the strinsg list using
append function
strings.append("welcome")
strings.append("to")
strings.append("Ethiopia")

print(numbers)
print(strings)
```

```
[1, 2, 3]
['welcome', 'to', 'Ethiopia']
```

```
numbers = []
strings = []
numbers.extend([1, 2, 3])
strings.extend(["welcome", "to", "Ethiopia"])
print(numbers)
print(strings)
```

```
[1, 2, 3]
['welcome', 'to', 'Ethiopia']
```

```
#given the following
ages = [16, 18, 20, ]
books = ['math', 'physics', 'bio']
#make a list ['math', 'physics', 'bio', 16, 18, 20] using .extend()
```

```
function
books.extend(ages)
books
```

```
['math', 'physics', 'bio', 16, 18, 20]
```

```
ages = [16, 18, 20, ]
books = ['math', 'physics', 'bio']
ages.extend(books)
ages
```

```
[16, 18, 20, 'math', 'physics', 'bio']
```

```
#given the following list
fruits = ['apple', 'mango', 'avocado']

#add 'banana' at index 1 and print the list, the list should loook
like ['apple', 'banana', 'mango', 'avocado'], hint use .insert()
function

fruits.insert(1, "banana")
fruits
```

```
['apple', 'banana', 'mango', 'avocado']
```

```
fruits.append("banana")
fruits
```

```
['apple', 'mango', 'avocado', 'banana']
```

```
fruits
```

Length of a List

we use the function `len(list_name)` to find the lenght of the list

length of list means the number of elements inside the list

```
x = [1, 2 ,3 ,5]
len(x)
```

```
4
```

```
len('hello')
```

```
5
```

```
y = ['hello', 3]
len(y) #what's gonna be the result?
```

```
2
```

Indexing

indexing mean **the position of elements in list**. we also have indexing in strings

`Note that`: ***`indexing starts from 0`*** (meaning the first item in the list is index 0)

For example: in the following list

mylist = [10, 178, 32, 85]

- so element 10 is at index 0
- 178 is at index 1
- 32 is at index 2
- 85 is at index 3

to get **32 from our list**, we can just call the index like this:

```
mylist[2]

l = [3, 10, 5]
#how can we get 5? what's the idnex of 5? 2
l[2]

5

l[0]

3
```

`Exercise`

```
#Find index of 5 in the following lists
l1 = [45, 5, 20, True]  # 1
l2 = [90, 20, 11, 'hello', 5] #4
l3 = [20, 65, 5, -12] #2

l1[1]

5

l2[4]

5

l3[2]

5
```

Negative Indexing

We can also do indexing starting from the last item and going from right to left.

`Note that:` ***`In this case, we begin from -1`***

For example: in the following list

mylist = [10, 178, 32, 85]

- so element 85 is at index -1
- 32 is at index -2
- 178 is at index -3
- 10 is at index -4

to get **32 from our list**, we can just call the index like this:

```
mylist[-2]

mylist = [2, 10, 5]
# if we want 5
print(mylist[-1])
print(mylist[2])
#in positive indexing it's index is 2
mylist[2]

5
5

5

#in negative indexing it's index is -1
mylist[-1]
```

Indexing list of list

it is exactly the same except the fact that we need to index more than once

for example: we have list l

```
l = [[1, 2], [3, 4]]
if we want 2 from the above list, how do we index?
l[0]? will give [1, 2] so we need to go one more step inside
inside the list [1, 2], 2 is at index 1 so we do:
l[0][1] #this means we will go to index 0 of our main list l and that
is the list [1, 2] and from this list we take 1st index

challenge = [[1, 2], [3, 4]] #it has two items
#so we want to find 3... it is in our second item... index 1
challenge[1]

[3, 4]

challenge[1] #gives us a list containing 3, [3, 4]
challenge[1][0]
```

```
3

m = [3, 4]
m[0]

3

l = [10, 15, [1, 2], [3, 4]] #4 items
#let's find 2
l[2]

[1, 2]

l[2][1]

2

l = [10, 40, [1, 2], 13, [3, 4]]



l[-1]

l[-1][-1] #what do we get?

l = [[1, 2], [3, 4]]
l[0], l[0][1] #what do we get?
```

we can index as many times as we want depending on the depth of our list:

for example: let's get 7 form the given list

```
l = [[1, 2], [3, 4], [5, 6, [7, 8]]]
now if we do l[2] we get [5, 6, [7, 8]]
and inside this list index 0 is 5, 1 is 6 and 2 is [7, 8]
so l[2][2] gives [7, 8]
did we find 7 yet? not really, so again we go inside the list [7, 8]
and we grap index 0 because it's at index 0

therefore to find 7, the indexing is l[2][2][0]

s = [[1, 2], [3, 4], [5, 6, [7, 8]]] # 3 items
# we want 7
s[2]

[5, 6, [7, 8]]

s[2][2]

[7, 8]

s[2][2][0]
```

```
7



l = [[1, 2], [3, 4], [5, 6, [7, 8]]]
l[0], l[1], l[2]
# len(l)
```

```
([1, 2], [3, 4], [5, 6, [7, 8]])
```

```
l[2]

l[2][2]

l[2][2][1]

l = [[1, 2], [3, 4], [5, 6, [7, 8]]]
l[2]

l[2][2]

l[2][2][0] # as you can see
```

```
Exercise
```

```
#Find index of 5 in the following lists .. one positive index and one
negative index
l1 = [45, 5, 20, True]  #
l2 = [90, 20, 11, 'hello', 5] #
l3 = [20, 65, 5, -12] #

# what is the output of the following?
l1[1]
l2[-2]
l3[2]
l1[-1]
l2[-4]

# fill the variable second_name with the second name from the names
list
names = ["John", "Eric", "Jessica"]
second_name =
second_name

# find 89 from the list below
myList = [[3, 5, 6], [[89], 7, 10]]

#change 7 from the list to 17
myList =  [[3, 5, 6], [[89], 7, 10]]
```

Finding the Index of an Item

we use the `.index() function` to get the index of an item

imagine we have a very long list, if you want to find **the index of the first occurance** of an item

for example:

```
myList = [3, 8, 32, 3, 5, 6, 7, 7, 73, 2, 12, 'hello', 'hi', True,
False, 21]
```

Now to find where 'hello' is we use

```
myList.index('hello')

myList = [3, 8, 32, 3, 5, 6, 7, 7, 73, 2, 12, 'hello', 'hi', True,
False, 21]
myList.index('hello')

11
```

Remeber `.index()` is going to return the index of the ***`FIRST OCCURENCE of the item if the item has duplicate`***

In the above example, we know that we have the item 3 repeated 2 times(at index 0 and 3), so when we use `.index()` it will give us the index of first occurence of 3(which is 0)

```
myList.index(3) #0, 3

0

myList.index(7) # 6, 7

6
```

Assigning value to an index

- we can easily replace or assign value in a given index by using
- `list_name[index] = value`

`Example:` Given the list

```
l = [1, 'hello', 0, 'world']
```

- imagine we want to change the value 1 to 2
- we can easily change the value at that index

```
l[0] = 2
```

as you can see we are assigning the value 2 at index 0 our list would look like

```
[2, 'hello', 0, 'world']

ls = [1, 5, 10]
ls.insert(1, 11) # [1, 11, 5, 10]
ls
```

`[1, 11, 5, 10]`

```
ls = [1, 5, 10] #to convert this to [1, 11, 10]
ls[1] = 11 #replace
ls
```

`[1, 11, 10]`

```
l = [1, 'hello', 0, 'world']
l[2] = 'Higa' #replacement
l
```

`[1, 'hello', 'Higa', 'world']`

```
l = [1, 'hello', 0, 'world']
l.insert(2, "higa") #[1, 'hello', "higa", 0, 'world']
l
```

`[1, 'hello', 'higa', 0, 'world']`

`Exercise`

```
mylist = [ 2, 3, 'Welcome', 5, 10]
#change the value 3 to 'hi'
mylist[1] = 'hi'
mylist
```

`[2, 'hi', 'Welcome', 5, 10]`

```
mylist = [ 2, 3, 'Welcome', 5, 10]
mylist.insert(1, 'hi')
mylist
```

`[2, 'hi', 3, 'Welcome', 5, 10]`

Slicing

- Slicing is *`breaking list into subpart by using indices`*
- To do slicing we use `[starting_index: end_index]`, remember **`the value at end_index is not included in the slice`**

`Example`

```
l = [10, 43, 2, 7]
l[0:2]
```

here we are slicing the values from index 0 to index 1 so we get `[10, 43]`

```
l = [10, 43, 2, 7]
l[0:2] #0, 1 stop
```

```
[10, 43]
```

```
x = [1, 'hello', 0, 'world']
x[1:3] #1, 2,  ['hello', 0]
```

```
['hello', 0]
```

```
evens = [i for i in range(2, 100, 2)]
len(evens)
```

```
49
```

```
evens
```

```
[2,
 4,
 6,
 8,
 10,
 12,
 14,
 16,
 18,
 20,
 22,
 24,
 26,
 28,
 30,
 32,
 34,
 36,
 38,
 40,
 42,
 44,
 46,
 48,
 50,
 52,
 54,
 56,
```

```
 58,
 60,
 62,
 64,
 66,
 68,
 70,
 72,
 74,
 76,
 78,
 80,
 82,
 84,
 86,
 88,
 90,
 92,
 94,
 96,
 98]
```

```
evens[4:8] #4, 5, 6, 7
```

```
[10, 12, 14, 16]
```

```
evens[4:] #from idnex 4 to the end
```

```
[10,
 12,
 14,
 16,
 18,
 20,
 22,
 24,
 26,
 28,
 30,
 32,
 34,
 36,
 38,
 40,
 42,
 44,
 46,
 48,
 50,
 52,
```

```
 54,
 56,
 58,
 60,
 62,
 64,
 66,
 68,
 70,
 72,
 74,
 76,
 78,
 80,
 82,
 84,
 86,
 88,
 90,
 92,
 94,
 96,
 98]
```

```
evens[:8] #from start to index 7
```

```
[2, 4, 6, 8, 10, 12, 14, 16]
```

if want everything ***from certain index up until the end***, we just use `[start_index:]`, we don't need to write the end_index

```
x = [1, 'hello', 0, 'world']
x[2:] # this means we want everything starting from index 2
```

```
[0, 'world']
```

if want ***from the beginning until certain index***, we use `[:end_index]`, we don't need to write the beginning

```
x[:2] #this means we want everything from the beginning till index 1
```

```
[1, 'hello']
```

`Exercise`

```
x = ['Welcome', 'To', 'Ethiopia', 'everyone', 1, 2, 6]
#how can we slice the above list to get only ['Ethiopia', 'everyone',
1, 2]
x[2:6]
```

```
['Ethiopia', 'everyone', 1, 2]

# We can also slice using negative indexing
x = [1, 'hello', 0, 'world']
x[-3:-1]

['hello', 0]

x[-2:] #from index -2 till the end

[0, 'world']

x[:-1] #from the beginning till index -2

[1, 'hello', 0]
```

Slicing with Skipping

- we can take slice of a list while skipping some steps
- Use the syntax `listname[start_index:end_index:step]`

for example:

```
s = [2, 5, 6 , 89, 10, 4, 11, 80]
s[3:7:2] # this means from index 3 to 6 by skipping 2 steps

s = [2, 5, 6 , 89, 10, 4, 11, 80] # 89, 4
s[3:7:2]

[89, 4]

s[-4:-1] #10, 4, 11

[10, 4, 11]
```

`Exercise`

```
#what will be the output
s[1:6:3]  #5, 10

[5, 10]
```

if the step is negative, it will go backwards

```
s = [2, 5, 6 , 89, 10, 4, 11, 80]
s[7:2:-2] #80, 4, 89

[80, 4, 89]

s[::-1] # it will go backwards from right to left

[80, 11, 4, 10, 89, 6, 5, 2]
```

```
s[::-2]
```

```
[80, 4, 89, 5]
```

```
Exercise
```

```
s = [2, 5, 6 , 89, 10, 4, 11, 80]
s[::-2] #80, 4, 89, 5
```

```
[80, 4, 89, 5]
```

```
example = [m for m in range(3, 20)]
```

```
example
```

```
[3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

```
example[3:7:2] #from index 3 to index 6, 6, 7, 8, 9
```

```
[6, 8]
```

```
example[-14:-10:2]
```

```
[6, 8]
```

```
example[7:3:-2]
```

```
[10, 8]
```

```
example[-10:3:-1]
```

```
[10, 9, 8, 7]
```

```
s[-10:3:-1] #since the step is negative, we are going from right to
left
```

5. More on Strings

strings are combination of characters

```
s = 'Hello world!'
```

Indexing

- Just like lists string characters are also indexed
- Indices start from 0

For example:

```
s = 'Ethiopia'
    E is at index 0
    t is at index 1
    h is at index 2
        .
        .
        .
    a is at index 7

s = 'Ethiopia'
s[2]

{"type":"string"}
```

Negative indexing

```
s[-3]

{"type":"string"}
```

Slicing

- The Rule is exactly the same as the one for Lists

```
s = 'hello'
s[:2] # all characters until index 1

{"type":"string"}

s[2:] # all characters after index 2

{"type":"string"}

s[1:4] # from index 1 to 3

{"type":"string"}
```

Slicing with Skipping

- we can take slice of a string while skipping specified steps
- Same syntax:

```
listname[start_index:end_index:step]

s = "Fantastic"
s[3:7:2] #this means from index 3 to 6 by skipping 2 steps

{"type":"string"}
```

`Exercise`

```
s = "Fantastic"
# what will be the output
s[1:8:3]
```

{"type":"string"}

if the step is negative, it will go backwards

```
s[::-1] # it will go backwards from right to left
```

{"type":"string"}

```
s[7:3:-2]
```

{"type":"string"}

Finding the index of a character in strings

- we can also use the `.index() function`for strings

```
s = 'Hello World'
s.index('o')
```

4

```
s.index('l') # we have 3 'l's however .index() will only give us the first ls index
```

2

Concatenation of Strings

- concatenation: **combining two or more strings using '+' operator**

```
first_name = 'John'
last_name = 'smith'
first_name + last_name #remember there will be no space inbetween
```

{"type":"string"}

```
# we can update the above one by adding space inbetween
first_name + " " + last_name
```

{"type":"string"}

6. String Formating

String Formatting with f-strings

```python name = "John" age = 23

print(f"Hello, {name}!") print(f"{name} is {age} years old.")

```
name = input("name: ")
print(name)

name: eyobed
eyobed

name = input("what your name? ")
print("hello " + name)
print("hello", name)

what your name? eyobed
hello eyobed
hello eyobed

name = input("What's your name? ")
print(f"Hello my name is {name}!")

What's your name? winner
Hello my name is winner!

name = input("what your name? ")
age = int(input("how old are you? "))
school = input("where do you sutdy? ")
color = input("what your favorite color? ")

#hello my name is "name". I am "age" year old. I study at "school" school. my favorite color is "color"

what your name? Behailu
how old are you? 17
where do you sutdy? Higa
what your favorite color? Red

print(f"Hello my name is {name}. I am {age} years old.")

Hello my name is Behailu. I am 17 years old.

print(f"I study at {school}")

I study at Higa

print(f"I study at {school} school. my favorite color is {color}")
```
```

```
I study at Higa school. my favorite color is Red
```

```python
first_name = "Sara"
last_name = "Abraham"
age = 16

# Combine them into a sentence
print(f"{first_name} {last_name} is {age} years old.")
```

```
Sara Abraham is 16 years old.
```

```python
# This prints out "John is 23 years old."
name = "John"
age = 23
print(f"{name} is {age} year old")
```

```
John is 23 year old
```

```python
# Format a float number to 1 or 2 decimal places
price = 4.5678
print(f"Price {price:.2f}")
```

```
Price 4.57
```

Exercise

```python
# Use f-string to print this message

# Given this data:
name = "John"
balance = 53.44

# Print: Hello John. Your current balance is $53.44.

# Write your code here:

print(f"Hello {name}. Your current balance is ${balance}.")
```

```
Hello John. Your current balance is $53.44.
```